

Coding Standard

Introduction

This standard defines how your code should be styled for assignments and projects. By following these conventions, you will produce code which is more readable and easier for you or someone else to follow. These guidelines apply to all programming languages. The examples are shown in Python. Being able to write clean legible code will also improve your employability.

The coding standard guidelines should also help you stay within the word count limits by including relevant information in your code instead of the written report.

*Certain assignment briefs such as those in Software Development in Practice may ask students to follow specific standard such **PEP-8**. **PEP-8** Cheat Sheet ([Python PEP Standards Cheat Sheet](#)) can be found near the end of this document.*

Identifiers

Variables/Attributes

Identifiers for all attributes and variables should be meaningful and if appropriate contain its own unit of measure. The unit of measure helps in understanding how to use it without reading the comments. Variables/attributes should always start with a lower case letter. Always use *camel case* to help aid reading of identifies, so an upper-case letter is introduced for every new word.

Examples:

```
weightInKilos    heightInMetres    delayInMilliseconds
```

Class Identifiers

Class identifiers always start with an uppercase letter and should always be a noun. So, Encryption is not a good name but Encryptor or EncryptionHelper are fine.

Examples:

```
Person    Doctor    Appointment
```

Constants

Identifiers of constants should always be expressed in uppercase, unless and only unless there is a necessary convention to use lowercase (for example to distinguish between g (acceleration due to earth's gravity and G the universal gravitational constant). For constants underscores are used to separate words.

Examples:

```
PI=3.149265445    COLUMNS=10
```

Method / Function Names

Method names should start with a lower case letter and then follow camel case. All method names should be verbs.

Examples:

```
add(a, b)
checkTwoEqual(array)
```

Comments

Your code should always contain comments that for a reasonably self-contained part of the code explain what the *intention* of that code is, what it is aiming to achieve. The comments should also provide an outline of the algorithm that is used to achieve that aim.

Besides that, the code should ideally be self-documenting, that is, it should be simple enough to be understood without having to refer to the comments. If the code is hard to follow then it should be simplified and in some cases broken up by the use of methods / functions to better structure the code.

Comments must always precede the part of the code that they refer to. Comments should not go over the right hand side of the edit window. If a comment is too long, then it should be broken down over several lines.

Pydoc

All comments in Python programs should follow the Pydoc format, this allows code documentation to be generated automatically.

Class Comments

All classes should start with a preamble describing the purpose of the class, what data it stores and what services it provides, so that a user of the class can quickly determine how to use the said class.

Method / Function Comments

At a minimum each public method in a class should be commented. This is vital since the public methods are the public interface to the code. Each method should have comments for both the input variables and the returned values (if there are any).

Example:

```
"""
    Calculates the factorial of a number
    @param Input integer to take the factorial of
    @return Factorial of input
"""
def factorial(n):
    result = 1
    for i in range(1, n + 1):
        result *= i
    return result
```

Use of On-line Resources and Textbooks

If in the preparation of your code you have used specific ideas or code fragments that you have found on-line or in a textbook, then you must include references to those sources in your comments and indicate clearly which part of the code is based on those. The reference must be precise:

- If you have used a textbook, then you must point to the specific page, pages, or figure that you have used.
- If you have used or adapted an answer to a question posted on some on-line forum, say, stackoverflow, then you must point to that specific answer, including a URL to that answer.
- If you have used or adapted a solution provided by some AI tool, say, OpenAI's ChatGPT or GitHub Copilot, then you must include the prompt or prompts that you have used.

If you have used a source just to gain an understanding of how a particular language construct works, then no reference to that source is required.

Examples:

```
"""
The code for the following function has been taken from
terminus (https://stackoverflow.com/users/9232/terminus):
Extracting minimum and maximum x-values Python
Stack Exchange Network, 11 Dec 2010.
https://stackoverflow.com/a/4416448 [accessed 14 Jan 2025].
User contributions licensed under cc by-sa 3.0
https://creativecommons.org/licenses/by-sa/3.0/
"""
def readfile(pathname):
    f = open(sti + '/testdata.txt')
    for line in f.readlines():
        line = line.strip()
        x, y = line.split(',')
        x, y = float(x),float(y)
        yield x, y

"""
The code for the following function has been obtained from
ChatGPT: Write a Python function that takes an array of strings as
argument and returns a sorted array as result but do not use the
built-in sort function.
OpenAI, ChatGPT based on GPT-4.
https://chat.openai.com/chat [accessed 14 Jan 2025].
"""
def sort_strings(arr):
    # Get the length of the array
    n = len(arr)

    # Perform bubble sort
    for i in range(n):
        # Flag to optimize: if no swaps, the array is already sorted
        swapped = False
```

```

    for j in range(0, n-i-1):
        # Compare adjacent strings
        if arr[j] > arr[j+1]:
            # Swap if out of order
            arr[j], arr[j+1] = arr[j+1], arr[j]
            swapped = True

    # If no two elements were swapped in the inner loop, then the array
    # is sorted
    if not swapped:
        break

    return arr

# Example usage:
arr = ["banana", "apple", "cherry", "kiwi"]
sorted_arr = sort_strings(arr)
print(sorted_arr) # Output: ['apple', 'banana', 'cherry', 'kiwi']

```

This attribution is required under the licenses that apply to the sources used in the examples and to the [Academic Integrity Policy](#) of the university.

Indentation and Braces

All codes must be indented in a consistent way to correctly convey the structure of the program. Unlike other programming languages, Python enforces the correct indentation and uses it to define blocks of code and indicate groups of statements.

Blocks of code that belong to the same structure (like if, for, while, def, etc.) must be indented the same amount. The level of indentation determines which statements are part of the block.

You must use spaces or tabs, but do not mix both in a single project or file. Python's official style guide (PEP 8) recommends using 4 spaces per indentation level. If you mix spaces and tabs, this can result in an `IndentationError`. If the indentation is inconsistent, Python will raise an `IndentationError`.

Blocks such as the body of statements if, for, while, def, and class must be indented.

Examples:

```

# If-else example
x = 10

if x > 5:
    print("x is greater than 5") # This line is part of the 'if' block
else:
    print("x is 5 or less") # This line is part of the 'else' block

def greet(name):
    print("Hello, " + name) # Indented inside the function body
    if name == "Alice":
        print("Welcome back, Alice!")
    else:

```

```
print("Welcome, guest!")  
  
greet("Alice")
```

Python PEP Standards Cheat Sheet

Section	Key Rules
Code Layout	- 4 spaces per indent (no tabs) - Max 79 chars/line (72 for docstrings) - Use blank lines to separate sections
Naming Conventions	- Modules/packages: lowercase_with_underscores - Classes/exceptions: PascalCase - Functions/variables: lowercase_with_underscores - Constants: ALL_CAPS - Private: <code>_var</code>, <code>__var</code> - Special: <code>__init__</code>, <code>__str__</code>
Imports	- One import per line - Order: <code>stdlib</code>, third-party, local - Avoid wildcard imports
Whitespace	- Spaces around operators: <code>x = y + 1</code> - No extra spaces inside <code>()</code>, <code>[]</code>, <code>{}</code> - Don't align with spaces, use indentation
Strings & Docstrings	- Consistent <code>'</code> or <code>"</code> - Triple quotes for docstrings - Docstrings: describe purpose, args, return
Comments	- Keep up to date - Use full sentences - Explain why, not what
Programming	- Use <code>is</code> for <code>None</code>: <code>if x is None</code> - Don't compare to <code>True/False</code> - Prefer <code>for</code> loops & comprehensions
Type Hints	- Use PEP 484 hints: <code>def add(x: int, y: int) -> int:</code>
Other PEPs	- PEP 20: Zen of Python - PEP 257: Docstrings - PEP 484: Type hints - PEP 572: Walrus operator

Conclusion

Following this guide will make your code easier to understand and follow and will help you debug and maintain your code.

The coding standard guidelines should also help you stay within the word count limits by including relevant information in your code instead of the written report.